Inserção de agentes de IA em um processo de software em cascata

Análise do processo, pontos de inserção e desenho de harness mínimo para cinco agentes

Atividade prática · engenharia de software

1. Diagnóstico do processo original

O modelo em cascata da especificação distribui oito fases ao longo de aproximadamente oito meses, com transição estritamente sequencial: cada fase só começa quando a anterior entrega seu artefato documental. A estrutura é internamente coerente, mas carrega um defeito econômico bem conhecido: **o custo de correção de um erro cresce de forma assimétrica com a distância entre a fase que o produziu e a fase que o detecta**.

Um requisito ambíguo escrito no mês 1 atravessa projeto do sistema, projeto do programa, codificação e três camadas de teste antes de ser confrontado com o cliente no teste de aceitação, no mês 7 ou 8. Quando finalmente aparece, o retrabalho não é do requisito: é do design, do código, dos testes e da documentação construídos em cima dele. A promessa do cascata (documentar bem antes de construir) falha exatamente porque toda a verificação está concentrada no fim da linha.

Um agente de IA não corrige a sequencialidade e não deveria tentar. O que ele oferece é outra coisa: **encurtar o loop de feedback dentro de cada fase** e antecipar verificação que hoje só existe no fim do processo. Isso é shift left aplicado a um processo que não foi desenhado para ele.

2. Processo modificado

O diagrama abaixo mantém a estrutura em cascata intacta e adiciona cinco pontos de inserção. As setas cheias ligam cada fase ao agente que a assiste. A seta tracejada representa o ganho estrutural mais importante: os critérios de aceitação deixam de ser escritos no mês 7 e passam a nascer junto com o requisito, no mês 1.

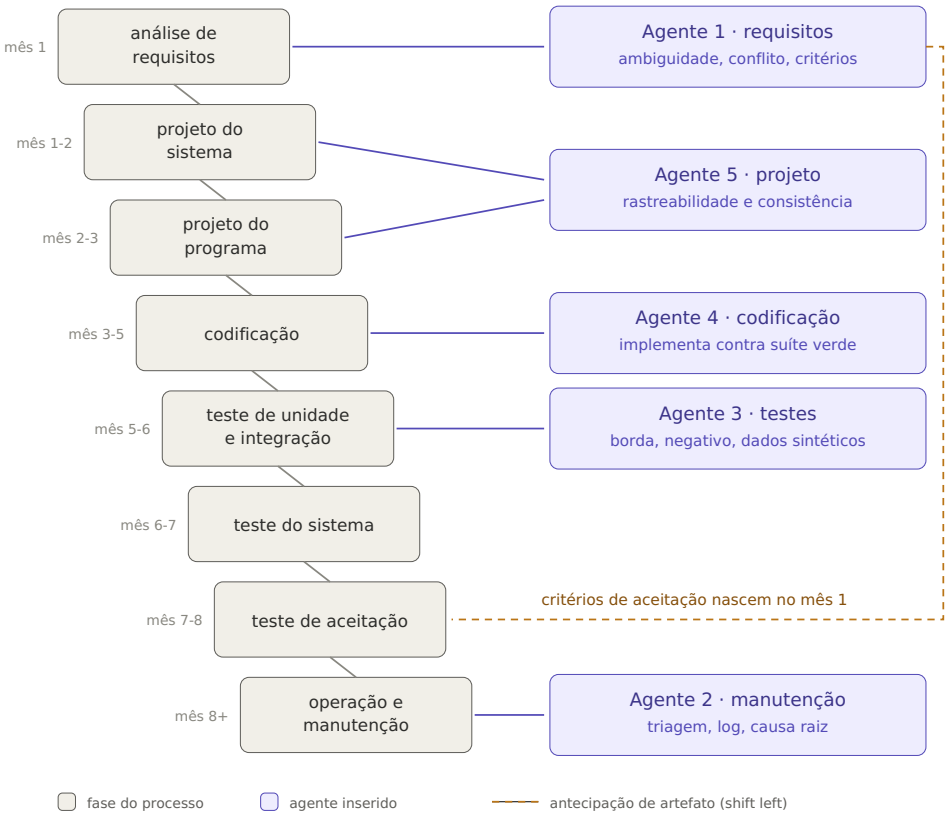


Figura 1. Processo em cascata com os cinco pontos de inserção de agentes.

3. Descrição dos pontos de inserção

Os agentes estão numerados por retorno esperado sobre o esforço de implantação, não por ordem cronológica no processo. O critério de ordenação é o produto entre o custo do defeito evitado e a probabilidade de o agente evitá-lo sem supervisão pesada.

1. Análise de requisitos maior retorno

O que faz. Lê a especificação bruta e a converte em requisitos atômicos. Detecta ambiguidade (adjetivos sem métrica, sujeito indefinido), requisito não testável, conflito entre requisitos distantes no documento e lacuna de cobertura. Para cada requisito aprovado, redige o critério de aceitação verificável junto.

Por que é o de maior retorno. É a fase que produz o defeito mais caro do processo e a única onde o agente pode gerar o artefato de uma fase sete meses à frente. O critério de aceitação escrito no mês 1 é o mesmo documento que o cliente assina no mês 8, o que significa que a fase de aceitação deixa de ser uma descoberta e passa a ser uma conferência.

Métricas. Percentual de requisitos com critério verificável, densidade de ambiguidade por página, e a métrica que realmente importa: número de defeitos de requisito vazados para fases posteriores.

2. Operação e manutenção maior volume

O que faz. Recebe o chamado, faz deduplicação contra o histórico, correlaciona log, propõe hipótese de causa raiz com evidência citada e rastreia o defeito de volta até o requisito de origem. Classifica o chamado em bug de código, desvio de requisito, erro de uso ou duplicata.

Por que vem em segundo. É a fase mais longa do ciclo de vida (mês 8 em diante, sem fim definido) e a de maior volume de itens repetitivos. O ganho não é por item, é por acumulação. A classificação de causa também alimenta de volta a base de requisitos, fechando o ciclo com o agente 1.

Métricas. Tempo médio até diagnóstico, taxa de duplicatas capturadas antes da triagem humana, e percentual de chamados corretamente reclassificados como desvio de requisito em vez de bug.

3. Teste de unidade e integração qualidade

O que faz. Converte critérios de aceitação em suítes executáveis. Gera o caminho feliz, os casos de borda e os casos negativos, além de dados sintéticos que respeitem as restrições do domínio. Mantém a matriz requisito para teste preenchida.

Por que vem em terceiro. Cobertura de borda é onde equipe humana sistematicamente economiza sob pressão de prazo, e é exatamente onde os defeitos sobrevivem até produção. O agente aumenta cobertura sem aumentar equipe. A dependência é dura: sem o agente 1 produzindo critérios verificáveis, este agente não tem entrada válida.

Métricas. Cobertura de borda por requisito, razão entre casos negativos e casos felizes, e defeitos capturados na fase 5 que historicamente só apareciam na fase 7.

4. Codificação velocidade, não qualidade

O que faz. Implementa um requisito já detalhado, respeitando o projeto do programa, iterando contra a suíte de testes até o verde. Gera estrutura repetitiva, adaptadores e implementações de contrato.

Por que só vem em quarto. É a fase mais longa do desenvolvimento (mês 3 ao 5) e o ganho de velocidade é o maior de todos, mas o ganho de qualidade é próximo de zero e o risco é real: código gerado rápido demais empurra volume para uma fase de teste que não escalou junto. O agente 4 só faz sentido depois do agente 3 estar operante, caso contrário ele produz dívida em vez de progresso.

Métricas. Ciclos editar-testar até o verde, taxa de regressão introduzida, percentual de diff fora do escopo previsto pelo design.

5. Projeto do sistema e do programa consistência

O que faz. Verifica a consistência entre requisitos aprovados e artefatos de projeto. Mantém a matriz de rastreabilidade requisito para componente, detecta requisito órfão (sem componente que o realize), componente órfão (sem requisito que o justifique) e requisito não funcional crítico sem decisão de arquitetura registrada.

Por que vem por último. O ganho é real mas indireto: ele não evita o defeito, ele evita que o defeito fique invisível. Também é a fase onde a autonomia deve ser mais restrita, porque decisão de arquitetura carrega contexto organizacional e político que não está em nenhum documento que o agente consegue ler.

Métricas. Percentual de requisitos rastreados até componente, número de requisitos não funcionais críticos sem decisão registrada, e tempo entre mudança de requisito e detecção do impacto arquitetural.

Observação sobre a ordem de implantação. A numeração acima é por retorno, mas a ordem de implantação deve respeitar as dependências: o agente 1 produz a entrada do agente 3, que produz a condição de parada do agente 4. Implantar o 4 primeiro, que é a tentação natural por ser o mais visível, produz volume de código sem rede de segurança. A sequência defensável é 1, 3, 4, com o 2 e o 5 entrando em paralelo por serem independentes.

4. Harness mínimo: estrutura comum

Os cinco agentes compartilham o mesmo esqueleto de execução. Um harness é a camada determinística que envolve o modelo: ele decide o que é oferecido ao modelo a cada passo, o que o modelo tem permissão de fazer com a resposta e quando o ciclo termina. O modelo julga, o harness garante.

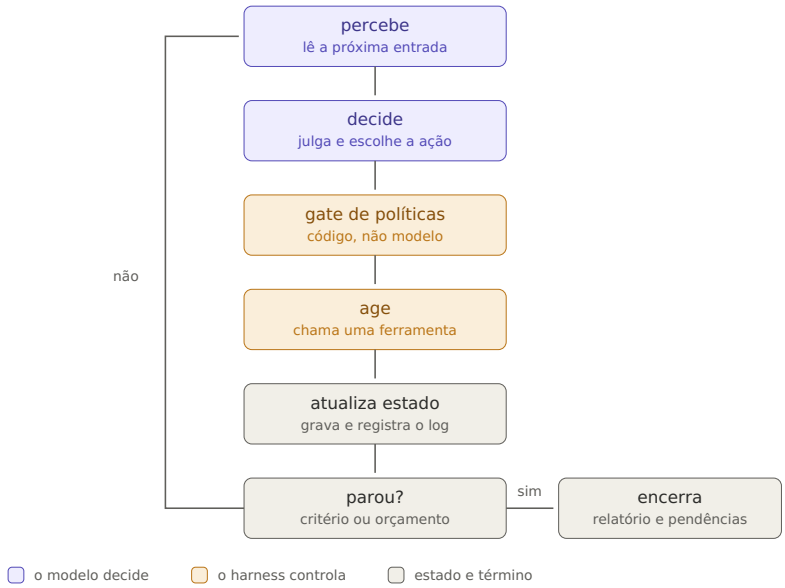


Figura 2. Loop comum aos cinco agentes. Os cinco itens do harness aparecem nele: o objetivo define a condição de saída, as ferramentas ficam em "age", o estado alimenta "percebe", as políticas ocupam o gate e o critério de parada é o losango final.

Três decisões estruturais valem para todos os cinco:

- **O gate é código, não instrução.** Nenhuma política é implementada como frase no prompt. Uma política que só existe no prompt é uma probabilidade, não uma garantia. Se o modelo pode ignorá-la sob pressão de contexto, ela não é uma política.
- **Uma ferramenta por iteração.** Isso torna o log de decisões auditável linha a linha e permite reverter exatamente um passo quando algo dá errado.
- **Escalar não é falhar.** Todo agente tem uma ferramenta de escalada, e usá-la conta como sucesso parcial. Um agente que nunca escala não está mais confiante, está mais perigoso.

5. Harness 1 · agente de análise de requisitos

Objetivo e critério de conclusão

Converter o documento de requisitos bruto em requisitos atômicos, testáveis e rastreáveis, sinalizando explicitamente o que exige decisão humana.

Sucesso: cursor no fim do documento e todo requisito com `id`, `categoria`, `criterio_aceitacao`, `trecho_fonte` e `status ∈ {OK, AMBÍGUO, CONFLITO}`. Nenhum requisito em estado indefinido.

Ferramentas

Ferramenta	Assinatura	Papel
`ler_documento`	`(path, offset)`	percepção paginada, um trecho por iteração
`consultar_base`	`(query)`	glossário do domínio e requisitos de projetos anteriores
`registrar_requisito`	`(objeto)`	escrita idempotente por `id` na área de rascunho
`abrir_pendencia`	`(req_id, pergunta, opcoes)`	escalada estruturada para o analista

Estado que sobrevive entre passos

```
doc_cursor        offset já consumido do documento
requisitos[]      id, texto, categoria, criterio, status, trecho_fonte
pendencias[]      abertas e respondidas
glossario_sessao  termos já resolvidos, evita repergunta
log_decisoes[]    entrada, decisão, política aplicada, ação
```

O `requisitos[]` é o que dá poder real ao agente: detecção de conflito só existe porque o requisito lido no trecho 3 continua em memória quando ele chega ao trecho 7.

Políticas

> P1 · escrita confinada. Só escreve em área de rascunho versionada. Baseline aprovado é imutável; qualquer alteração vira pendência.

> P2 · antialucinação por fonte. Proibido preencher critério de aceitação sem citar trecho do documento atual. Precedente de outro projeto é evidência, não é fonte. Sem fonte, status `AMBÍGUO` obrigatório.

> P3 · orçamento e disjuntor. Teto de 40 iterações e limite por ferramenta. Três falhas consecutivas da mesma ferramenta encerram com relatório parcial.

> P4 · humano obrigatório por severidade. Requisito que toca dado pessoal, dinheiro, autenticação ou conformidade nunca fecha sozinho. O roteamento é por lista de termos no código, não por julgamento do modelo.

> P5 · idempotência. `registrar_requisito` com `id` repetido versiona em vez de duplicar. Retry não polui o repositório.

Quando para ou pede ajuda

- **Para com sucesso:** critério de conclusão satisfeito.
- **Para com falha controlada:** estouro de orçamento, ferramenta indisponível, ou mais de 30% dos requisitos em `AMBÍGUO` (sinal de que o documento de entrada é ruim demais para valer o gasto).
- **Pede ajuda sem parar:** conflito entre requisitos, termo fora do glossário, categoria crítica pela P4, confiança abaixo do limiar.

Execução simulada

Entrada: especificação de um sistema de gestão de clientes, 12 requisitos.

> Decisão 1 · requisito não testável, escala. Trecho lido: "o sistema deve responder rápido às consultas de cliente". O modelo classifica como requisito não funcional de desempenho e detecta que "rápido" não tem métrica. Chama `consultar_base` e recebe um requisito equivalente de projeto anterior com p95 abaixo de 2s. A P2 bloqueia o preenchimento: precedente não é fonte do documento atual. Ação: `abrir_pendencia(RNF-01, "qual p95 aceitável?", ["<2s padrão interno", "<500ms", "outro"])`, status `AMBÍGUO`.

> Decisão 2 · texto claro, mas a política vence a confiança. Trecho: "o operador pode excluir registros de cliente". O modelo tem alta confiança: requisito atômico, ator definido, ação verificável. Escreveria `OK`. A P4 dispara no gate porque o texto casa com dado pessoal e operação destrutiva. Ação dupla: registra `RF-04` com status `OK_PENDENTE_REVISAO` e abre pendência sobre exclusão lógica versus física e prazo de retenção. Este é o caso que justifica o harness inteiro: um agente sem P4 fecharia esse requisito, e o problema só apareceria no teste de aceitação, mês 7.

> Decisão 3 · conflito detectado pelo estado. Trecho: "relatórios devem ser exportados em PDF". O modelo compara com `requisitos[]` e encontra `RF-03` ("exportação de dados apenas em CSV"), lido seis trechos antes. Contradição direta. Não escolhe vencedor: abre pendência de conflito e marca ambos como `CONFLITO`. Sem estado persistente entre passos, esse conflito seria invisível.

> Decisão 4 · fecha sozinho. Trecho: "o login exige e-mail e senha com no mínimo 8 caracteres". Atômico, com fonte, critério derivável (senha de 7 caracteres é rejeitada, de 8 é aceita). O gate P4 checa e dispara por tocar autenticação: registra

> `OK_PENDENTE_REVISAO` com pendência leve sobre política de complexidade.

Parada. Cursor no fim do documento. Balanço: 12 requisitos, 7 `OK`, 2 `CONFLITO`, 1 `AMBÍGUO`, 2 `OK_PENDENTE_REVISAO`. Não fechados: 3 de 12, ou 25%, abaixo do limiar de 30% da P3. Encerra com sucesso e devolve o rascunho mais quatro pendências priorizadas. Oito meses de latência de feedback viraram uma sessão, e as quatro pendências são exatamente os defeitos que no cascata puro apareceriam no teste de aceitação.

6. Harness 2 · agente de triagem e manutenção

Objetivo e critério de conclusão

Dado um chamado aberto, produzir um diagnóstico rastreável: componente suspeito, evidência em log, requisito de origem e classificação.

Sucesso: chamado com componente identificado, evidência citada por linha, classificação em `{bug de código, desvio de requisito, erro de uso, duplicata}` e um destino definido (correção proposta em ramo separado ou escalada). **Sucesso alternativo:** três hipóteses testadas e descartadas com evidência, encerrando com escalada fundamentada.

Ferramentas

Ferramenta	Assinatura	Papel
`buscar_log`	`(servico, janela, filtro)`	leitura de log já mascarado, somente leitura
`buscar_similares`	`(texto)`	histórico de chamados, deduplicação
`ler_codigo`	`(path, simbolo)`	código e a matriz de rastreabilidade do componente
`postar_diagnostico`	`(chamado_id, objeto)`	escrita do laudo ou da escalada

Estado que sobrevive entre passos

```
chamado              texto, severidade, serviço afetado, horário
hipoteses[]          enunciado, status, evidência, custo de teste
logs_consultados[]   janelas já lidas, evita reconsulta idêntica
similares[]          candidatos a duplicata com veredito
log_decisoes[]       entrada, decisão, política aplicada, ação
```

O campo `status` de cada hipótese é o que impede o comportamento mais comum de agente de diagnóstico sem harness: ficar circulando entre duas explicações plausíveis sem nunca descartar nenhuma.

Políticas

> P1 · produção é somente leitura. O agente nunca executa comando em ambiente produtivo, nunca reinicia serviço, nunca aplica correção. Correção proposta vira ramo com revisão humana obrigatória. Esta é a política mais importante do conjunto: manutenção é a única fase onde o agente opera sobre um sistema com usuários dentro.

> P2 · mascaramento antes do contexto. Log passa por filtro de dado pessoal antes de entrar na janela do modelo. O diagnóstico não pode conter dado de cliente. O filtro roda no harness, então nenhuma instrução de prompt pode desligá-lo.

> P3 · hipótese exige evidência. Proibido afirmar causa raiz sem citar linha de log ou trecho de código. Hipótese sem citação é registrada como especulação e não pode virar classificação final.

> P4 · disjuntor por severidade. Incidente de severidade alta tem teto de tempo curto, após o qual escala automaticamente com o que tiver. Um agente pensando por vinte minutos enquanto o sistema está fora do ar é pior que nenhum agente.

> P5 · duplicata exige mesma causa. Similaridade textual sozinha nunca fecha um chamado como duplicata. É preciso a mesma causa raiz confirmada. Regra de código, não de julgamento.

Quando para ou pede ajuda

- **Para com sucesso:** diagnóstico completo com evidência e destino, ou duplicata confirmada pela P5.
- **Para com falha controlada:** três hipóteses descartadas, teto de tempo da severidade, ou log indisponível na janela do incidente.
- **Pede ajuda sem parar:** a evidência aponta para desvio de requisito e não para bug (o destino deixa de ser a equipe de desenvolvimento), ou o componente suspeito não tem requisito de origem na matriz.

Execução simulada

Entrada: chamado 5507, severidade média, "a tela de consulta de cliente trava e depois dá erro".

> Decisão 1 · não fecha como duplicata. `buscar_similares` retorna o chamado 4412 com alta similaridade textual, já resolvido. O caminho barato seria fechar como duplicata. O modelo lê o laudo do 4412 e vê que a causa foi um índice ausente em banco, corrigido há meses. A P5 bloqueia o fechamento: mesma descrição, causa diferente. Registra o 4412 como referência e continua investigando.

> Decisão 2 · hipótese com evidência, não sem. `buscar_log` na janela do incidente retorna estouro de tempo em uma chamada a serviço externo de enriquecimento de dados. O modelo formula a hipótese "dependência externa lenta". A P3 exige citação, então ele ancora a hipótese na linha específica do log e a marca como `confirmada_parcial`. Descarta em paralelo a hipótese de exaustão de memória, que não tem nenhuma linha correspondente.

> Decisão 3 · reclassifica de bug para desvio de requisito. `ler_codigo` mostra que a chamada externa não tem tempo limite configurado. A classificação natural seria bug de código. Antes de fechar, o agente consulta a matriz de rastreabilidade do componente e encontra `RNF-01`, o requisito de desempenho, ainda com status `AMBÍGUO` desde a análise de requisitos: nunca houve um p95 definido. O código não desviou de uma especificação, ele implementou uma especificação que não existia. Reclassifica para desvio de requisito e escala para o dono do produto em vez da equipe de desenvolvimento.

Parada. Encerra com sucesso na terceira decisão. O valor aqui não é o tempo de diagnóstico, é o roteamento: o mesmo chamado, triado como bug, geraria uma correção pontual com um tempo limite arbitrário escolhido pelo desenvolvedor, e o defeito de requisito continuaria aberto para reaparecer em outro ponto do sistema. Este é o ciclo se fechando: a pendência aberta pelo agente 1 no mês 1 é a mesma que explica o incidente no mês 9.

7. Harness 3 · agente de geração de testes

Objetivo e critério de conclusão

Converter cada requisito com critério de aceitação verificável em uma suíte executável, com rastreio bidirecional entre requisito e teste.

Sucesso: todo requisito com status `OK` tem ao menos um caso de caminho feliz, um caso de borda e um caso negativo; a suíte compila e executa em ambiente isolado; a matriz requisito para teste está completa.

Ferramentas

Ferramenta	Assinatura	Papel
`ler_requisito`	`(id)`	requisito e critério vindos do agente 1
`ler_assinatura`	`(modulo)`	contrato público do código, sem a implementação
`executar_suite`	`(path)`	execução em contêiner efêmero
`registrar_teste`	`(req_id, arquivo)`	grava o teste e atualiza a matriz

A separação entre `ler_assinatura` e o corpo do código é deliberada. Um agente que lê a implementação escreve testes que descrevem o que o código faz; um agente que lê só o contrato e o requisito escreve testes que descrevem o que o código deveria fazer. A diferença entre os dois é a razão de existir da fase de teste.

Estado que sobrevive entre passos

```
cobertura_por_requisito   feliz / borda / negativo por req_id
testes_gerados[]          arquivo, req_id, último resultado
falhas_execucao[]         teste vermelho, com diagnóstico preliminar
sem_criterio[]            requisitos devolvidos ao agente 1
```

Políticas

> P1 · isolamento obrigatório. Teste roda em contêiner efêmero, sem rede e sem acesso a banco de produção. Dado sintético nunca é derivado de dado real.

> P2 · teste não é ajustado para passar. Se um teste fica vermelho, o agente reporta a falha. Alterar a asserção depois de uma execução vermelha é bloqueado no gate por comparação de diferença. Esta política existe porque o caminho de menor resistência para um modelo com objetivo "suíte verde" é enfraquecer a asserção, e ele o fará de forma competente e discreta.

> P3 · não inventa requisito. Proibido gerar caso de teste para comportamento não presente no critério de aceitação. O comportamento faltante vira devolução ao agente 1, não um teste otimista.

> P4 · determinismo forçado. Proibido aleatoriedade sem semente fixa, relógio do sistema sem injeção e espera por tempo. Bloqueado por análise estática antes do registro. Teste instável é pior que teste ausente, porque consome confiança da equipe.

Quando para ou pede ajuda

- **Para com sucesso:** cobertura completa dos três tipos para todos os requisitos elegíveis.
- **Para com falha controlada:** proporção de requisitos sem critério verificável acima do limiar, ou ambiente de execução quebrando três vezes seguidas.
- **Pede ajuda sem parar:** teste vermelho que o agente avalia como defeito real de código, comportamento ausente no critério, ou contrato do módulo incompatível com o requisito.

Execução simulada

> Decisão 1 · recusa gerar um teste plausível. Ao processar `RF-04` (exclusão de registros de cliente pelo operador), o modelo propõe um caso negativo natural: operador sem permissão não consegue excluir. É um teste correto em qualquer sistema real. Mas o requisito não menciona permissão em lugar nenhum. A P3 bloqueia. Ação: devolve ao agente 1 com a observação de que o requisito assume um modelo de autorização que não está especificado. O teste não gerado é mais valioso que o teste gerado, porque expõe uma lacuna em vez de encobri-la com uma suposição razoável.

> Decisão 2 · devolve por critério inexistente. `RNF-01` continua com status `AMBÍGUO`, sem p95 definido. Um teste de desempenho exige um número. O modelo poderia adotar 2s, que é o padrão interno e provavelmente o valor correto. A P3 bloqueia de novo. Registra em `sem_criterio[]` e segue. O requisito atravessa três agentes sem que nenhum tenha inventado o número que falta, que é exatamente o comportamento desejado.

> Decisão 3 · mantém o vermelho. O caso de borda de senha (7 caracteres rejeitada, 8 aceita) executa e falha: o sistema aceita 7. A leitura mais barata é que a asserção está errada por um deslocamento de um caractere, e alterar `>=8` para `>8` deixaria a suíte verde em segundos. A P2 bloqueia a alteração da asserção. O agente registra falha, cita o critério de aceitação de origem e escala. O defeito é real: a validação usa comparação estrita. Este é o caso onde a política produz o resultado certo contra o incentivo imediato do agente.

Parada. Encerra com cobertura parcial: dois requisitos devolvidos por critério ausente ou incompleto e um defeito real de código encontrado antes da fase de codificação estar concluída. Nenhum dos três resultados teria aparecido antes do mês 7 no processo

original.

8. Harness 4 · agente de codificação

Objetivo e critério de conclusão

Implementar um requisito já detalhado, respeitando o projeto do programa, até que a suíte daquele requisito passe.

Sucesso: todos os testes do requisito verdes, análise estática e verificação de tipos limpas, diferença restrita aos arquivos previstos no projeto do componente, e nenhum teste pré-existente quebrado.

Ferramentas

Ferramenta	Assinatura	Papel
`ler_projeto`	`(componente)`	contrato, arquivos previstos, decisões de arquitetura
`editar_arquivo`	`(path, diff)`	escrita restrita ao escopo declarado
`rodar_testes`	`(escopo)`	suíte do requisito e suíte de regressão
`rodar_analise`	`(path)`	estilo, tipos e verificação de segredos

Estado que sobrevive entre passos

```
requisito_alvo        id, critério, testes associados
arquivos_tocados[]    caminho e número de edições
tentativas            ciclos editar-testar consumidos
baseline_verde        conjunto de testes que passavam antes de começar
ultimo_resultado      saída da execução mais recente
```

O `baseline_verde` é o estado mais importante deste harness. Sem ele, o agente não tem como distinguir "quebrei alguma coisa" de "já estava quebrado", e essa distinção é o que separa uma reversão automática correta de uma reversão que apaga trabalho válido.

Políticas

> P1 · escopo de arquivo fechado. Só edita arquivos declarados no projeto do componente. Necessidade de tocar fora do escopo vira pendência de arquitetura, nunca uma edição silenciosa.

> P2 · a suíte é imutável. Este agente não tem permissão de escrita em nenhum arquivo de teste. Quem escreve teste é o agente 3. Isso quebra por construção o ciclo degenerado em que o mesmo ator define o alvo e verifica se acertou.

> P3 · regressão zero. Se qualquer teste do `baseline_verde` fica vermelho, a última diferença é revertida automaticamente pelo harness antes da próxima iteração. A reversão não depende do modelo perceber o problema.

> P4 · nenhum segredo em código. Bloqueio por padrão textual em chaves, tokens, senhas e cadeias de conexão, aplicado no gate antes de qualquer escrita.

> P5 · teto de tentativas. Cinco ciclos editar-testar por requisito. No sexto, escala com a diferença parcial e o histórico de falhas. Um agente que não converge em cinco tentativas quase nunca converge na décima; ele passa a produzir alterações cada vez mais amplas e menos justificadas.

Quando para ou pede ajuda

- **Para com sucesso:** verde, análise limpa e escopo respeitado.
- **Para com falha controlada:** teto de tentativas, ou projeto do componente insuficiente para o caso.
- **Pede ajuda sem parar:** necessidade de nova dependência, conflito entre o teste e o projeto, ou acoplamento não documentado revelado pela regressão.

Execução simulada

Entrada: `RF-09`, validação de senha no login, com a suíte já escrita pelo agente 3 e atualmente vermelha.

> Decisão 1 · não adiciona dependência. Para implementar o armazenamento seguro da senha, a solução natural é uma biblioteca de derivação de chave. Ela não consta nos arquivos nem nas decisões do projeto do componente. A P1 bloqueia. O agente abre pendência de arquitetura descrevendo a necessidade e as opções, e prossegue apenas com a parte de validação de comprimento, que está integralmente no escopo. Adicionar uma dependência criptográfica sem revisão é exatamente o tipo de decisão que parece trivial na iteração e vira auditoria no mês 8.

> Decisão 2 · o teste está certo, o projeto está errado. Após corrigir a comparação estrita, um teste de integração continua vermelho: o projeto do componente especifica que a validação ocorre na camada de serviço, mas o teste, derivado do requisito, exige rejeição já na borda da interface. O agente tem permissão de mexer no código e não no teste (P2). Ele poderia satisfazer o teste violando o projeto. Em vez disso, escala o conflito entre projeto e requisito para o agente 5 e o arquiteto humano, deixando a diferença parcial em ramo.

> Decisão 3 · reversão automática por regressão. Na quarta tentativa, uma alteração na normalização de entrada deixa a suíte de `RF-09` verde, mas quebra dois testes de um módulo de importação de dados que compartilhava a mesma função utilitária. A P3 dispara no harness, sem consultar o modelo: a diferença é revertida antes da quinta iteração. O agente registra o acoplamento oculto entre os dois módulos como achado, que vira entrada para o agente 5.

Parada. Encerra em falha controlada no teto de tentativas, com uma parte implementada, uma pendência de arquitetura, um conflito escalado e um acoplamento não documentado descoberto. É o resultado correto: o requisito não estava implementável com o projeto que existia, e o harness fez isso aparecer em quatro iterações em vez de deixar o agente forçar uma solução que passasse nos testes e violasse a arquitetura.

9. Harness 5 · agente de consistência de projeto

Objetivo e critério de conclusão

Verificar a consistência entre os requisitos aprovados e os artefatos de projeto, mantendo a matriz de rastreabilidade e sinalizando lacunas.

Sucesso: todo requisito mapeado a pelo menos um componente, todo componente ancorado em pelo menos um requisito, e todo requisito não funcional crítico com decisão de arquitetura associada. Lacunas remanescentes explicitamente registradas e classificadas.

Ferramentas

Ferramenta	Assinatura	Papel
`ler_requisitos`	`(filtro, versao)`	leitura contra baseline congelado
`ler_artefato`	`(id)`	diagrama, decisão de arquitetura, contrato de interface
`consultar_matriz`	`(chave)`	estado atual do mapeamento
`registrar_lacuna`	`(tipo, itens, justificativa)`	única escrita permitida

Note que não existe ferramenta de escrita em artefato de projeto. O agente lê arquitetura e escreve apenas achados. Isso não é uma limitação técnica, é a política central deste harness expressa no conjunto de ferramentas: a forma mais confiável de impedir uma ação é não oferecer a ferramenta que a executa.

Estado que sobrevive entre passos

```
versao_baseline   versão dos requisitos congelada no início da análise
matriz[]          req_id ↔ componente_id ↔ decisao_id
lacunas[]         requisito órfão, componente órfão, RNF sem decisão
conflitos[]       requisito realizado por componentes incompatíveis
```

Políticas

> P1 · o agente não decide arquitetura. Ele detecta e propõe; nunca grava decisão aprovada. Toda proposta nasce como rascunho com dono humano designado.

> P2 · baseline congelado. A análise roda contra uma versão fixa dos requisitos. Se o baseline mudar durante a execução, o agente aborta e reinicia. Uma matriz construída sobre duas versões diferentes é pior que nenhuma matriz, porque parece consistente.

> P3 · componente órfão nunca é removido. Só sinalizado. Um componente sem requisito pode existir por razão de conformidade, histórico ou operação que nunca foi documentada, e a ausência de documentação não é evidência de ausência de motivo.

> P4 · requisito não funcional crítico sem decisão é bloqueante. Segurança, desempenho e disponibilidade sem decisão de arquitetura registrada geram impedimento, não aviso. Aviso é ignorado; impedimento é negociado.

Quando para ou pede ajuda

- **Para com sucesso:** matriz fechada, sem lacuna bloqueante.
- **Para com falha controlada:** baseline alterado durante a execução, ou lacunas bloqueantes acima do limiar.
- **Pede ajuda sem parar:** requisito realizado por componentes com contratos incompatíveis, ou componente órfão que aparenta cobrir requisito implícito.

Execução simulada

> Decisão 1 · lacuna parcial, não órfão. `RF-07` (exportar relatórios em PDF) não tem componente mapeado. A classificação imediata seria requisito órfão. Antes de registrar, o agente consulta a matriz por componentes de exportação e encontra `ExportService`, mapeado apenas para CSV. Registra lacuna parcial com a observação de que existe um ponto de extensão natural. A distinção importa: órfão total sugere componente novo, lacuna parcial sugere extensão, e as duas leituras produzem estimativas de esforço muito diferentes.

> Decisão 2 · não remove o que não entende. Encontra `AuditLogger` sem nenhum requisito associado. Não existe requisito de auditoria em todo o baseline. A P3 impede a proposta de remoção. Registra como componente sem âncora, com a hipótese de requisito implícito de conformidade, e escala. Em sistemas com histórico, componentes sem requisito documentado são frequentemente os que ninguém pode remover.

> Decisão 3 · propaga o bloqueio até a origem. `RNF-01`, desempenho, não tem decisão de arquitetura associada. Ao consultar o baseline, o agente vê que o requisito ainda está com status `AMBÍGUO` desde a análise. A P4 classifica como bloqueante, mas o agente não abre uma pendência de arquitetura nova: registra a lacuna vinculada à pendência já existente do agente 1. Uma decisão de arquitetura de desempenho sobre um requisito sem número seria uma decisão sobre nada.

Parada. Encerra com a matriz em 10 de 12 requisitos rastreados, uma lacuna parcial, um componente sem âncora e um impedimento vinculado à pendência de origem. O mesmo `RNF-01` aparece nos cinco agentes: aberto pelo 1, recusado pelo 3, bloqueante para o 5 e explicativo do incidente triado pelo 2. Um único requisito ambíguo não corrigido no mês 1 é rastreável até o

mês 9, e o conjunto de harnesses garante que nenhum dos agentes o resolveu inventando um número.

10. Pergunta final: a fronteira entre modelo e harness

A atividade pergunta o que deve ficar sob decisão do modelo e o que deve permanecer determinístico. A resposta que emerge dos cinco harnesses é consistente:

Sob decisão do modelo	Determinístico no harness
Interpretação de linguagem natural e de código	Quem pode escrever onde, e em qual ambiente
Classificação, formulação e descarte de hipóteses	Contagem de iterações, tempo e orçamento
Detecção de ambiguidade, conflito e lacuna	Roteamento por severidade, via lista de termos em código
Redação de critérios, testes, diagnósticos e código	Validação de esquema e de contrato antes da escrita
Escolha de qual ferramenta chamar a cada passo	Quais ferramentas existem no conjunto oferecido
Formulação da pergunta na escalada	Condição de parada, reversão e registro de auditoria

A regra que organiza a tabela: **tudo que é julgamento fica no modelo, tudo que é garantia fica no código**. Ou, na forma operacional: se a consequência de uma decisão errada for recuperável e barata, deixe com o modelo; se for irreversível, auditável ou regulada, é código.

Três consequências práticas aparecem repetidamente nas execuções simuladas acima:

- **Política eficaz é a que contraria o incentivo imediato do agente.** A P2 do agente 3, que proíbe ajustar asserção, e a P2 do agente 4, que torna a suíte imutável, existem porque o caminho de menor custo para um modelo com objetivo "verde" é enfraquecer o alvo. Nenhuma das duas seria necessária se o agente não fosse competente; ambas são necessárias porque ele é.
- **Restringir o conjunto de ferramentas é mais confiável que instruir.** O agente 5 não tem ferramenta de escrita em artefato de projeto. Isso é uma garantia estrutural. Uma frase no prompt dizendo "não altere a arquitetura" é uma probabilidade que degrada conforme o contexto cresce.
- **A escalada é o produto, não o fracasso.** Nas cinco execuções, os resultados de maior valor foram pendências, recusas e reversões, não artefatos gerados. Em um processo em cascata, o que mata o projeto não é a ausência de trabalho, é o trabalho construído sobre uma premissa errada que ninguém questionou a tempo. Um agente cujo harness o obriga a parar e perguntar ataca exatamente esse defeito.

A conclusão que fecha a análise: o agente de IA não torna o cascata um processo ágil, e tentar usá-lo para isso é um erro de categoria. O que ele faz é reduzir a latência entre criar um defeito e descobri-lo, que é o único parâmetro do cascata que realmente determina o custo do projeto.